

Computer architecture is the **design and organization of a computer system**—basically how all the hardware components work together to execute programs.

◆ Key Components of Computer Architecture

1. CPU (Central Processing Unit)

- The “brain” of the computer
- Executes instructions
- Has two main parts:
 - **ALU (Arithmetic Logic Unit)** → performs calculations
 - **Control Unit** → directs operations

2. Memory

- Stores data and instructions
- Types:
 - **Primary memory** (RAM, cache) → fast, temporary
 - **Secondary memory** (HDD, SSD) → slower, permanent

3. Input/Output Devices

- Input: keyboard, mouse
- Output: monitor, printer

4. Buses

- Communication pathways between components
- Types:
 - Data bus
 - Address bus
 - Control bus

◆ Types of Computer Architecture

1. Von Neumann Architecture

- Data and instructions share the same memory
- Simple but can cause bottleneck (Von Neumann bottleneck)

2. Harvard Architecture

- Separate memory for data and instructions
- Faster, used in embedded systems

◆ Important Concepts

• Instruction Set Architecture (ISA)

Defines the commands a CPU can execute (e.g., x86, ARM)

• Pipelining

Multiple instructions processed at different stages simultaneously

- **Cache Memory**
Small, very fast memory close to CPU
- **Parallelism**
Multiple tasks executed at the same time (multi-core processors)

◆ Why It Matters

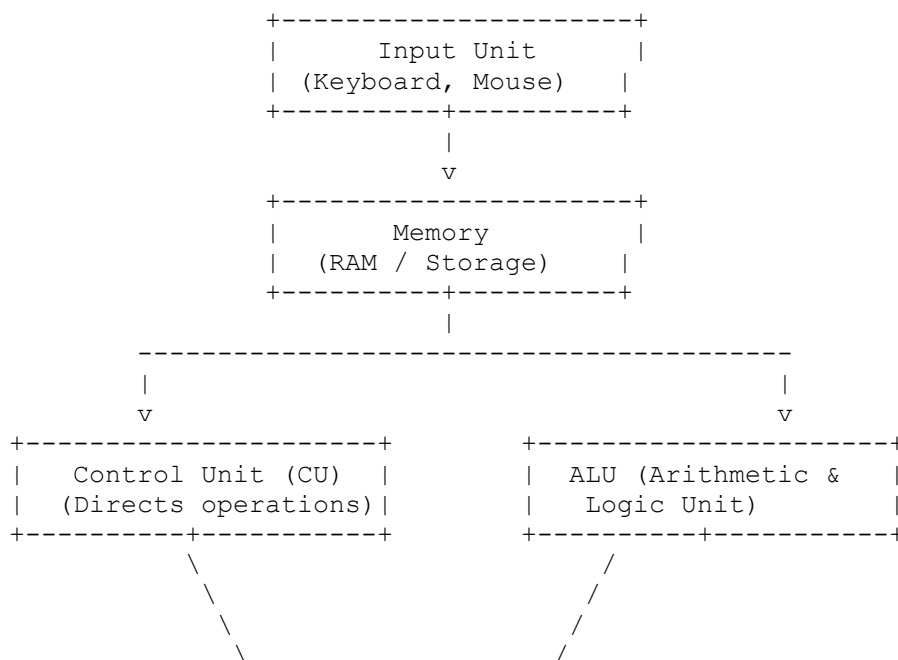
Computer architecture determines:

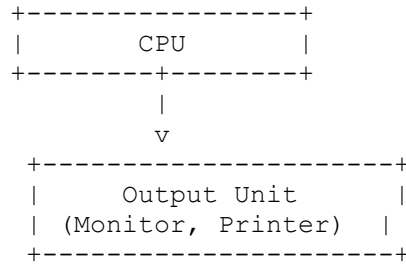
- Performance (speed)
- Power efficiency
- Cost
- Compatibility with software

If you want, I can:

- Explain it in **very simple (exam-style)** notes
- Provide **diagrams**
- Or go deeper into topics like ****pipelining, cache, or RISC vs CI**

Here's a **simple and clear diagram of Computer Architecture (Von Neumann model)** 





□ How it works (quick understanding)

1. Input is given → goes to **Memory**
2. **CPU (CU + ALU)** processes instructions
3. Results are stored back in memory
4. Output is shown via output devices

If you want, I can also give:

- 📄 **Harvard architecture diagram**
- ⚡ **Pipelining diagram (important for exams)**
- □ **Short notes + diagrams for Wipro prep**

The “**5 pillars of computer architecture**” usually refer to the fundamental principles used to design efficient computer systems (commonly taught based on modern CPU design concepts).

🔖 5 Pillars of Computer Architecture

1. Abstraction

- Hiding complex details and showing only essential features
- Example: You use Java without knowing hardware-level operations

☞ Makes system design simpler and manageable

2. Performance

- Focus on making programs run faster

- Measured using:
 - Execution time
 - Throughput

☞ Techniques: pipelining, caching, parallelism

3. Parallelism

- Performing multiple tasks at the same time
- Types:
 - Instruction-level
 - Data-level
 - Thread-level

☞ Example: Multi-core processors

4. Pipelining

- Breaking instruction execution into stages
- Multiple instructions processed simultaneously (like an assembly line)

☞ Improves CPU efficiency and speed

5. Memory Hierarchy

- Organizing memory in levels based on speed and cost:
 - Registers (fastest)
 - Cache
 - RAM
 - Hard disk (slowest)

☞ Helps balance performance and cost

📖 Easy Trick to Remember

☞ **A P P P M**
Abstraction

Performance
Parallelism
Pipelining
Memory hierarchy

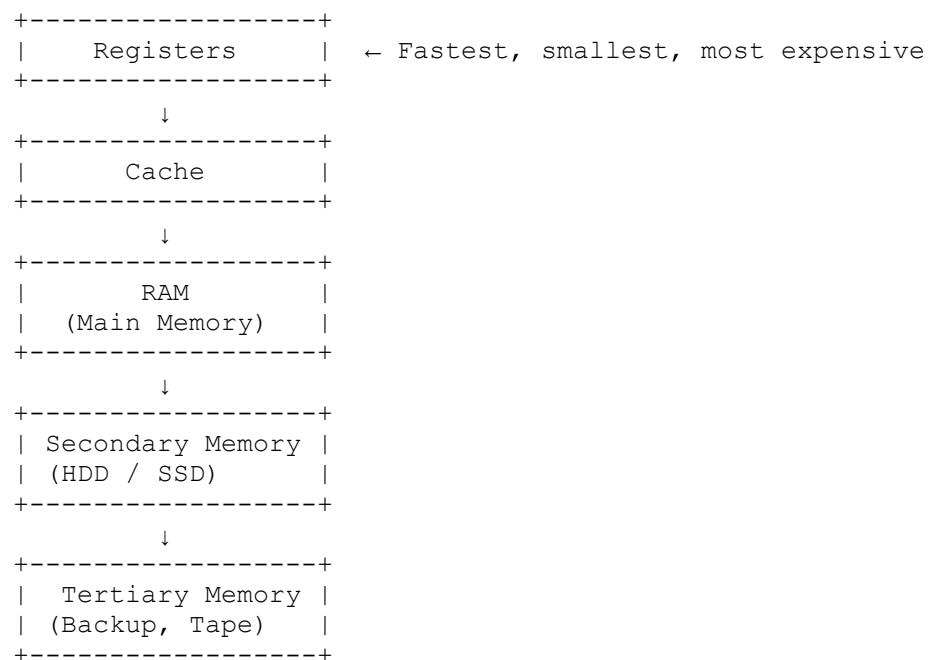
Exam One-Liner

☞ The 5 pillars of computer architecture are abstraction, performance, parallelism, pipelining, and memory hierarchy, which together help design efficient and high-speed systems.

If you want, I can give:

-  Diagram for memory hierarchy
- ⚡ Short notes for quick revision
- ☐ MCQs for Wipro exam practice

Here's a **clear diagram of Memory Hierarchy** 



↑ Speed decreases ↓
↑ Cost per bit decreases ↓
↑ Size (capacity) increases ↓

□ Quick Understanding

- **Top** → **Fast, expensive, small size**
 - **Bottom** → **Slow, cheap, large size**
-

🔥 Easy Way to Remember

👉 **R → C → R → S → T**

(Registers → Cache → RAM → Secondary → Tertiary)

👑 Exam Tip

👉 Always draw it as a **pyramid or vertical structure** and mention:

- Speed ↓
 - Cost ↓
 - Size ↑
-

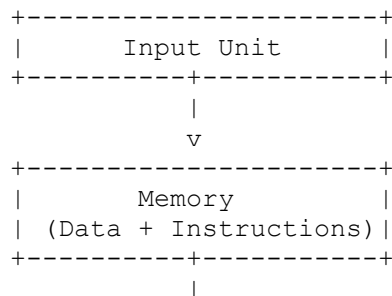
If you want, I can also:

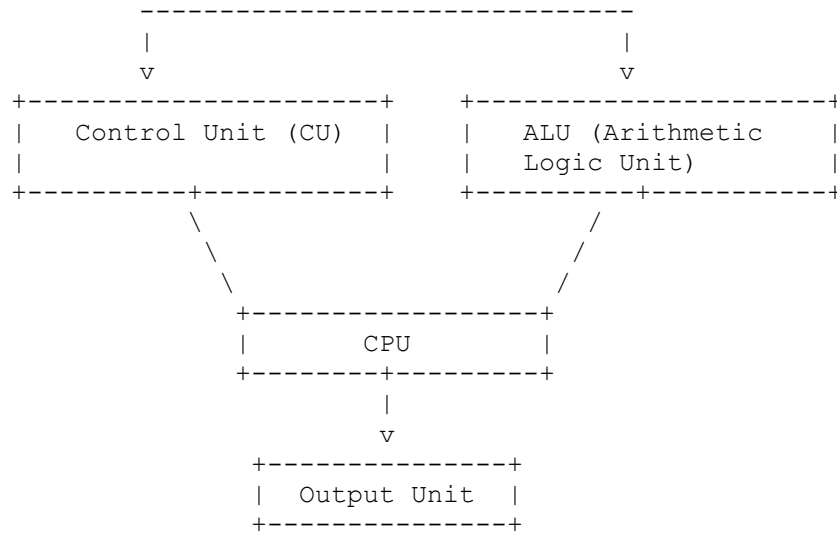
- Convert this into a **handwritten-style diagram (for exams)**
- Give **short notes + MCQs for Wipro prep**

📄 Von Neumann Architecture

Von Neumann Architecture is a computer design model where **data and instructions are stored in the same memory** and share the same pathway.

📊 Diagram





◆ Key Features

- ☐ **Single Memory**
Same memory stores both instructions and data
- **↻ Sequential Execution**
Instructions are executed one after another
- **‡ Single Bus System**
One pathway for both data and instructions

◆ How It Works

1. Input is given
2. Stored in memory (along with instructions)
3. CPU fetches instruction
4. ALU processes it
5. Result goes to output

⚠ Limitation (Important for Exams)

🔑 Von Neumann Bottleneck

- Data and instructions share the same bus
 - Causes **slow performance** due to traffic
-

🔍 Advantages

- Simple design
 - Cost-effective
 - Easy to implement
-

✖ Disadvantages

- Slower due to bottleneck
 - Cannot fetch data & instruction simultaneously
-

📝 One-Line Answer

📖 *Von Neumann architecture is a computer design where both data and instructions are stored in the same memory and processed sequentially.*

If you want, I can also:

- Compare **Von Neumann vs Harvard (important question)**
- Give **short notes + MCQs for Wipro exams**

💻 Development of Computer Architecture

The development of computer architecture shows how computers evolved from simple machines to powerful modern systems.

📁 1. First Generation (1940s–1950s)

- Technology: **Vacuum tubes**
 - Examples: ENIAC, UNIVAC I
 - Features:
 - Very large and expensive
 - High power consumption
 - Machine language only
-

⚙ 2. Second Generation (1950s–1960s)

- Technology: **Transistors**
- Features:
 - Smaller, faster, more reliable
 - Assembly language introduced
 - Less heat generation

3. Third Generation (1960s–1970s)

- Technology: **Integrated Circuits (ICs)**
- Examples: IBM System/360
- Features:
 - Higher speed
 - Operating systems introduced
 - Multiprogramming

💡 4. Fourth Generation (1970s–Present)

- Technology: **Microprocessors (VLSI)**
- Examples: Intel 4004
- Features:
 - Personal computers (PCs)
 - High speed, small size
 - GUI, networking

5. Fifth Generation (Present & Future)

- Technology: **Artificial Intelligence & Parallel Processing**
- Features:
 - AI-based systems
 - Machine learning
 - Natural language processing

Simple Timeline Diagram

Vacuum Tubes → Transistors → ICs → Microprocessors → AI Systems
(1st) (2nd) (3rd) (4th) (5th)

🔗 Quick Trick to Remember

🔑 V T I M A

(Vacuum → Transistor → IC → Microprocessor → AI)

📌 Exam One-Liner

🔑 *Computer architecture has evolved from vacuum tube-based systems to modern AI-based architectures through five generations, improving speed, size, and efficiency.*

If you want, I can:

- Give **comparison table of all generations (very important for exams)**
- Provide **MCQs for Wipro preparation**

✂ RISC vs CISC (Computer Architecture)

RISC and **CISC** are two types of Instruction Set Architectures (ISA).

📊 Comparison Table

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instructions	Simple & few	Complex & many
Execution Time	Fast (1 clock cycle)	Slower (multiple cycles)
Instruction Length	Fixed	Variable
Memory Usage	More (needs more instructions)	Less (complex instructions do more work)
Hardware	Simple	Complex
Pipelining	Easy to implement	Difficult
Examples	ARM, MIPS	x86

📄 Simple Diagram

RISC:

```
ADD R1, R2, R3
SUB R4, R5, R6
MOV R7, R8
```

(Simple steps)

CISC:

```
ADD A, B
MUL A, B
DIV A, B
```

(Complex single instruction)

💡 Key Idea

- **RISC** → Breaks tasks into many simple instructions
 - **CISC** → Uses fewer but more powerful instructions
-

🔥 Advantages

✓ RISC

- Faster execution
- Better pipelining
- Power efficient

✓ CISC

- Less code required
 - Easier for programmers
 - Efficient memory use
-

✗ Disadvantages

✗ RISC

- More instructions needed
- Larger program size

✗ CISC

- Slower due to complex instructions
 - Harder to pipeline
-

🔖 Easy Trick to Remember

👉 **RISC = Simple & Fast**

👉 **CISC = Complex & Powerful**

📖 Exam One-Liner

👉 *RISC uses simple and fewer instructions for faster execution, while CISC uses complex instructions to reduce the number of instructions per program.*

If you want, I can:

- Give **real-life examples (ARM vs Intel)**
- Provide **MCQs asked in Wipro exams**

⚙️ Pipeline Processing

Pipeline processing is a technique where multiple instructions are executed **simultaneously in different stages**, like an assembly line.

🏢 Basic Pipeline Stages

Instruction 1: IF → ID → EX → MEM → WB

Instruction 2: IF → ID → EX → MEM → WB

Instruction 3: IF → ID → EX → MEM → WB

- **IF** → Instruction Fetch
 - **ID** → Instruction Decode
 - **EX** → Execute
 - **MEM** → Memory access
 - **WB** → Write Back
-

📌 Key Idea

👉 While one instruction is executing, another is being decoded, and another is being fetched.

✓ Advantages

- Increases CPU throughput
- Faster execution of multiple instructions

✗ Disadvantages

- Pipeline hazards:
 - Data hazard
 - Control hazard
 - Structural hazard

🚀 Superscalar Architecture

Superscalar means executing **multiple instructions in the same clock cycle** using multiple processing units.

📊 Simple Diagram

Cycle 1: Instruction 1 → ALU1
 Instruction 2 → ALU2

Cycle 2: Instruction 3 → ALU1
 Instruction 4 → ALU2

📌 Key Idea

👉 Instead of one instruction per cycle, CPU executes **2 or more instructions per cycle**.

💧 Difference (Important)

Feature	Pipeline Processing	Superscalar
Concept	Overlapping stages	Multiple instructions at once
Execution	One per stage	Many per cycle
Hardware	Single pipeline	Multiple execution units
Goal	Increase throughput	Increase parallelism

📖 Easy Understanding

- **Pipeline** → Like a **factory assembly line**
 - **Superscalar** → Like **multiple workers doing tasks at the same time**
-

📌 Exam One-Liner

☞ *Pipelining improves performance by overlapping instruction execution stages, while superscalar architecture executes multiple instructions simultaneously using multiple processing units.*

If you want, I can also:

- Give **diagram-based questions for practice**
- Explain **pipeline hazards with examples (very important for exams)**

⚠ Pipeline Hazards

Pipeline hazards are problems that **prevent the next instruction from executing in the correct clock cycle** in a pipeline.

☞ In simple words: *anything that slows down or stops smooth pipelining*

💡 Types of Pipeline Hazards

1. Data Hazard

- Happens when an instruction depends on the result of a previous instruction

📌 Example

I1: $R1 = R2 + R3$

I2: $R4 = R1 + R5$ ← depends on I1

☞ Problem: I2 needs R1 before I1 finishes

✓ *Solution*

- Forwarding (data bypassing)
 - Pipeline stall (waiting)
-

2. Control Hazard (Branch Hazard)

- Occurs due to **branch instructions** (if, loop, jump)

 Example

```
if (x > 0)
    instruction A
else
    instruction B
```

☞ CPU doesn't know which path to take immediately

✓ *Solution*

- Branch prediction
 - Delayed branching
-

3. Structural Hazard

- Happens when hardware resources are **not sufficient**

 Example

- Two instructions need the same memory at the same time

☞ Conflict over hardware usage

✓ *Solution*

- Add more hardware
 - Use scheduling
-

Summary Table

Hazard Type	Cause	Example
Data Hazard	Dependency between instructions	Using previous result
Control Hazard	Branching instructions	if-else, loops
Structural Hazard	Resource conflict	Same hardware used

Easy Trick to Remember

DCS

- **D** → Data
 - **C** → Control
 - **S** → Structural
-

Exam One-Liner

 Pipeline hazards are situations that cause delays in pipelined execution, mainly classified as data, control, and structural hazards.

If you want, I can also:

- Give **numerical problems on pipelining**
- Provide **MCQs asked in Wipro exams**

Arithmetic & ALU Design (Computer Architecture)

◆ What is Arithmetic Unit?

The **Arithmetic Unit** performs **mathematical operations** such as:

- Addition (+)
- Subtraction (−)
- Multiplication (×)

- Division ($\div$)

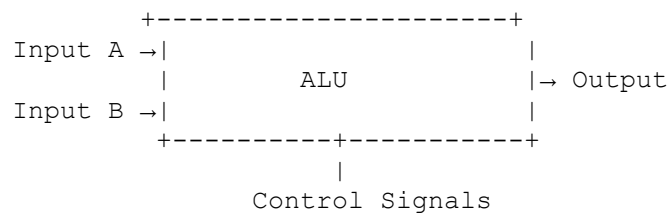
It is a part of the **ALU**.

◆ What is ALU (Arithmetic Logic Unit)?

The **ALU** is a **core component of the CPU** that performs:

- □ Arithmetic operations (add, subtract, etc.)
 - 🔍 Logical operations (AND, OR, NOT, XOR)
-

📊 Basic ALU Block Diagram



◆ Components of ALU Design

1. **Registers**
 - Store input data (A, B)
 2. **Adder Circuit**
 - Performs addition (basic building block)
 3. **Logic Circuit**
 - Performs AND, OR, NOT, XOR
 4. **Multiplexer (MUX)**
 - Selects which operation result to output
 5. **Control Unit Signals**
 - Decide which operation ALU performs
-

◆ Arithmetic Operations Design

➤ Binary Addition (using Full Adder)

- Combines bits with carry

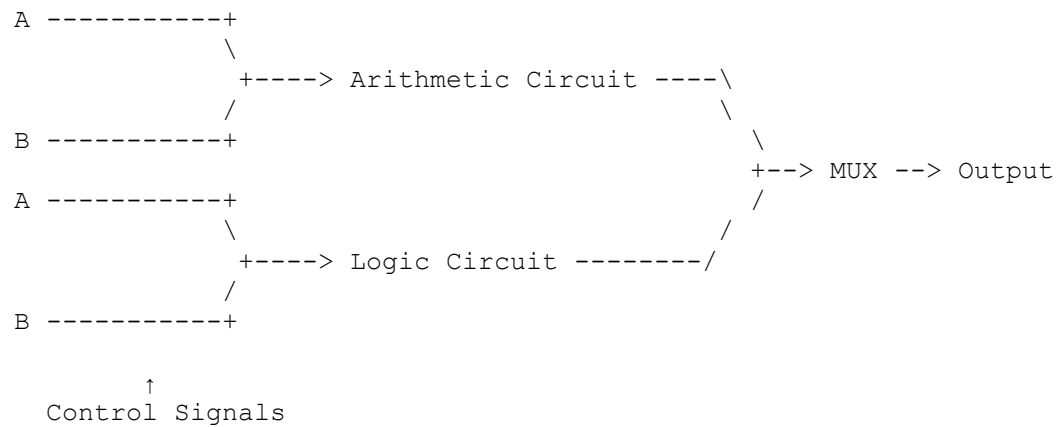
➤ Subtraction

- Done using **2's complement**
☞ $A - B = A + (2\text{'s complement of } B)$
-

🔗 Logic Operations

- AND → both inputs must be 1
 - OR → at least one input is 1
 - NOT → reverses bit
 - XOR → different inputs → 1
-

📊 Simple ALU Internal Design



🔥 Key Points

- ALU is the **heart of CPU operations**
 - Uses **binary arithmetic**
 - Controlled by **control unit signals**
 - Works with **registers and memory**
-

🔗 Easy Trick

☞ **ALU = Arithmetic + Logic**

Exam One-Liner

☞ *ALU is a digital circuit that performs arithmetic and logical operations using combinational circuits like adders, logic gates, and multiplexers.*

If you want, I can:

- Explain **half adder & full adder with diagrams (very important)**
- Give **numerical problems on binary arithmetic**
- Provide **Wipro exam MCQs**

Virtual Memory & Paging

◆ What is Virtual Memory?

Virtual Memory is a memory management technique where the system uses **secondary storage (hard disk)** as an extension of **RAM**.

☞ It allows you to run large programs even if RAM is limited.

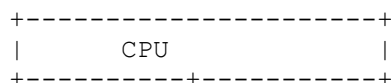
□ Key Idea

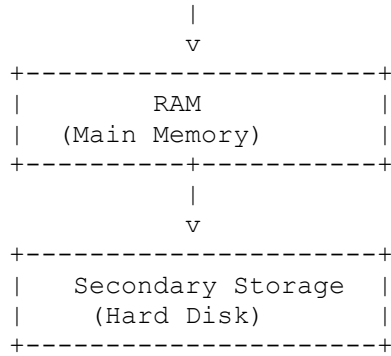
- Not all program data is loaded into RAM at once
 - Only required parts are loaded → rest stays on disk
-

✓ Advantages

- Run large programs
 - Efficient memory usage
 - Supports multitasking
-

Virtual Memory Diagram





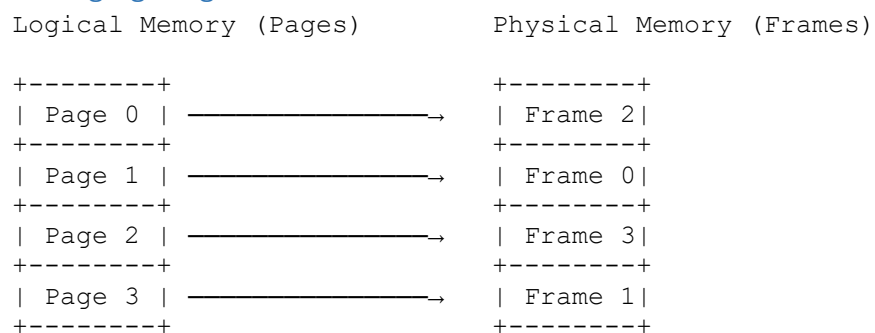
◆ What is Paging?

Paging is a technique used to implement virtual memory by dividing memory into fixed-size blocks.

□ Key Terms

- **Page** → Fixed-size block of logical memory
- **Frame** → Fixed-size block of physical memory (RAM)
- **Page Table** → Maps pages to frames

▢ Paging Diagram



◆ How Paging Works

1. CPU generates a **logical address**
2. Page number is checked in **page table**
3. Mapped to a **frame in RAM**
4. Data is accessed

🔥 Advantages of Paging

- Eliminates **external fragmentation**
- Efficient memory usage
- Easy memory allocation

✖ Disadvantages

- **Internal fragmentation**
- Extra memory needed for page table
- Slightly slower due to address translation

🔗 Easy Trick

👉 **Virtual Memory = RAM + Disk**

👉 **Paging = Divide into Pages & Frames**

📌 Exam One-Liner

👉 *Virtual memory allows execution of large programs using disk space as extra memory, and paging divides memory into fixed-size pages and frames for efficient management.*

If you want, I can also:

- Solve **numerical problems on paging (very important)**
- Explain **segmentation vs paging**
- Give **Wipro exam MCQs**

Your question seems like an **exam-type MCQ** 🖱

Let me clarify it in a simple way:

? “In which memory allocation is memory space occupied?”

☞ The correct concept here is:

✓ Contiguous Memory Allocation

💡 Explanation

In **contiguous memory allocation**:

- A process is given a **continuous (single block) of memory**
- That entire block is **occupied/reserved** for that process

☞ So, memory gets **fully occupied in one place**

📊 Example

```
+-----+
| Process A | ← occupies continuous memory
+-----+
| Process B |
+-----+
| Free Space |
+-----+
```

💧 Why this is the answer?

- Memory is **allocated as one block**
 - Once allocated → that portion is **occupied**
 - No splitting like paging/segmentation
-

✗ Other Types (for clarity)

- **Paging** → memory divided into pages (not contiguous)
 - **Segmentation** → divided into segments
 - These do NOT occupy one single continuous block
-

Exam One-Liner

☞ *In contiguous memory allocation, a process occupies a continuous block of memory.*

If you want, I can give:

- MCQs based on this (Wipro pattern)
- Difference between **contiguous vs non-contiguous allocation**

Your question seems like an **exam-type MCQ** 🐼
Let me clarify it in a simple way:

? “In which memory allocation is memory space occupied?”

☞ The correct concept here is:

✓ **Contiguous Memory Allocation**

◆ Explanation

In **contiguous memory allocation**:

- A process is given a **continuous (single block) of memory**
- That entire block is **occupied/reserved** for that process

☞ So, memory gets **fully occupied in one place**

Example

Process A	← occupies continuous memory
Process B	
Free Space	

🔥 Why this is the answer?

- Memory is **allocated as one block**
 - Once allocated → that portion is **occupied**
 - No splitting like paging/segmentation
-

✗ Other Types (for clarity)

- **Paging** → memory divided into pages (not contiguous)
 - **Segmentation** → divided into segments
 - These do NOT occupy one single continuous block
-

👉 Exam One-Liner

☞ *In contiguous memory allocation, a process occupies a continuous block of memory.*

If you want, I can give:

- MCQs based on this (Wipro pattern)
- Difference between **contiguous vs non-contiguous allocation**

📊 Benchmarking & Performance Metrics (Computer Architecture)

💡 What is Benchmarking?

Benchmarking is the process of **measuring and comparing the performance of a computer system** using standard test programs.

☞ It helps to know **how fast or efficient a system is**

☐ Example

- Running programs like:
 - SPEC benchmark

- Geekbench

These test CPU, memory, etc.

◆ Types of Benchmarks

1. Synthetic Benchmarks

- Artificial programs
- Example: SPEC CPU

2. Application Benchmarks

- Real-world programs
 - Example: games, browsers
-

◆ What are Performance Metrics?

Performance metrics are the **parameters used to evaluate system performance**.

📊 Important Performance Metrics

1. Execution Time (Most Important)

- Time taken to run a program

☞ Less time = Better performance

2. Throughput

- Number of tasks completed in a given time

☞ More throughput = Better system

3. Clock Rate (CPU Speed)

- Number of cycles per second (GHz)

☞ Higher clock rate $\neq$ always better (depends on CPU design)

4. CPI (Cycles Per Instruction)

$$\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Number of Instructions}}$$

☞ Lower CPI = better performance

5. CPU Execution Time

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

☞ Main formula for numericals

🔥 Relationship

- Performance $\propto 1 / \text{Execution Time}$
☞ Less time $\rightarrow$ Higher performance
-

🧠 Easy Trick

☞ **Performance = Speed + Efficiency**

👉 Exam One-Liner

☞ *Benchmarking evaluates system performance using standard tests, while performance metrics like execution time, CPI, and throughput measure efficiency.*

If you want, I can:

- Solve **numerical problems on CPI & execution time**
- Give **Wipro exam MCQs**
- Provide **short revision notes (1-page)**

✂ 4 GHz AMD vs 4 GHz Intel — Which is Faster?

☞ **Short Answer:**

Clock speed (4 GHz) alone does NOT decide performance.
Either **AMD or Intel can be faster** depending on architecture.

📖 Why 4 GHz ≠ Same Performance

Even if both CPUs run at **4 GHz (4 billion cycles/sec)**, performance depends on:

◆ **1. IPC (Instructions Per Cycle)**

- How many instructions CPU executes in one clock cycle
- ☞ Higher IPC = better performance

✓ If AMD has higher IPC → AMD is faster

✓ If Intel has higher IPC → Intel is faster

◆ **2. Architecture Design**

- Pipeline depth
- Cache size
- Branch prediction
- Core efficiency

☞ Modern CPUs differ a lot internally

◆ **3. Cores & Threads**

- More cores = better multitasking
 - Example:
 - 4-core vs 8-core → 8-core wins in multitasking
-

◆ **4. Cache Memory**

- Larger cache → faster data access

Simple Comparison Idea

CPU Performance $\neq$ Just GHz

Performance = IPC $\times$ Clock Speed $\times$ Efficiency

Real Insight (Important for Exams)

- Earlier: Intel had higher IPC $\rightarrow$ faster
- Now: AMD Ryzen often matches or beats Intel

☞ So:

Same GHz $\neq$ Same Speed

Easy Analogy

☞ GHz = speed of a worker

☞ IPC = work done per step

Two workers running at same speed:

- One does more work per step $\rightarrow$ **faster overall**
-

Exam One-Liner

☞ *Between 4 GHz AMD and Intel processors, the faster one depends on IPC, architecture, and core efficiency, not just clock speed.*

If you want, I can:

- Compare **AMD Ryzen vs Intel Core (real examples)**
- Give **MCQs asked in Wipro exams**

How Fast Can RAM Feed the CPU?

The speed at which RAM feeds the CPU is called **memory bandwidth**.

◆ 1. Basic Idea

☞ RAM does **not send data continuously at CPU speed**

☞ Speed depends on:

- RAM frequency (MHz)
- Bus width (bits)
- Number of channels (single/dual)

📊 Memory Bandwidth Formula

$$\text{Bandwidth} = \text{Memory Speed} \times \text{Bus Width}$$

☞ (More accurately: also depends on transfers per cycle)

◆ 2. Example (DDR4 RAM)

- DDR4-3200 RAM
- 64-bit bus (8 bytes)

☞ Bandwidth $\approx$
 $3200 \times 8 = \mathbf{25.6 \text{ GB/s}}$

◆ 3. What Affects RAM Speed?

✓ Frequency (MHz)

- Higher MHz $\rightarrow$ faster data transfer

✓ Channels

- Single channel $\rightarrow$ slower
- Dual channel $\rightarrow \sim 2\times$ faster

✓ Latency (CL)

- Lower latency $\rightarrow$ quicker response

🔗 4. CPU vs RAM Speed

- CPU works in **GHz (very fast)**
- RAM is **much slower**

☞ That's why we use:

- **Cache memory (L1, L2, L3)**
→ to bridge the speed gap

📊 Real Flow

CPU (fastest)
↑
Cache (very fast)
↑
RAM (moderate)
↑
Storage (slow)

🔥 Key Point

☞ RAM cannot fully keep up with CPU speed

☞ So performance depends on:

- Cache efficiency
- Memory bandwidth
- Latency

🔗 Easy Understanding

- CPU = super fast worker
- RAM = helper bringing data
- Cache = assistant standing next to CPU

📌 Exam One-Liner

☞ RAM feeds the CPU at a rate determined by memory bandwidth, which depends on memory speed, bus width, and channels, but is slower than CPU speed.

If you want, I can:

- Solve **numerical problems on bandwidth**
- Explain **cache vs RAM (very important)**
- Give **Wipro exam MCQs**